

It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

Developers routinely attribute the per-request cost of JSON Web Token (JWT) validation to cryptography, and optimise or relocate it on that assumption. We test the assumption empirically in `jsonwebtoken`, a widely used Node.js library. The signature primitive is 2.208 μ s of a 29.92 μ s validation. The dominant component, 18.04 μ s (60%), is a discarded key-parse attempt: a shared secret supplied as a string is first offered to the asymmetric key parser and is resolved only after that parse fails. Its price is set by a transitive dependency, the OpenSSL release bundled with the runtime: issued from C, the same calls cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. In a containerised service, validation adds 384.6 μ s of processor time per request, within 0.8% of the microbenchmark prediction. Of seven JWT libraries in four languages, only this one attempts the parse, so the cost follows a design choice rather than JWT validation as such. The obvious optimisation is unsafe: the parse entered the library with a key-confusion security fix, and removing it naively fails the library's own tests. We give a narrow fix that passes the full test suite and makes validation 4.15 times faster, and identify pre-parsed keys

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

as the remedy that removes the cost for every input. In our sample of 327 public call sites, none pre-parses the key. We derive guidance for developers, library authors and performance engineers, and release a replication package.

Keywords: Empirical study, performance analysis, software libraries, root-cause analysis, JSON Web Token, OpenSSL

1. Introduction

Performance engineering depends on attributing cost correctly. When a cost is believed to belong to one component and belongs to another, optimisation effort goes to the wrong place, and architectural remedies are adopted for problems that could have been fixed in a few lines. Empirical studies of performance bugs find that many stem from inefficient library usage and avoidable redundant work, and that the fixes are small once the cause is located (Jin et al., 2012; Selakovic and Pradel, 2016). This paper reports such a case on a path that runs on every request of many web services: the validation of JSON Web Tokens (JWTs) (Jones et al., 2015).

The usual reasoning is intuitive: validating a token means verifying a signature, signatures are cryptography, and cryptography is expensive, so the cost should be *moved* to a sidecar proxy, a gateway or dedicated hardware. The assumption has been tested before one layer down, where Coarfa et al. (2006) found RSA a minority of TLS server cost. We test it at the application layer, in `jsonwebtoken`, a widely used Node.js library, and find it wrong. A second natural explanation, that key material is converted from a string into a key object on every call, fails as well: the conversion costs $0.583\ \mu\text{s}$.

The actual mechanism is a control-flow accident (Figure 2). Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed, so every validation constructs and discards an OpenSSL parse failure, at $18.04\ \mu\text{s}$ of a $29.92\ \mu\text{s}$ call. Its price depends on which OpenSSL release performs it, by more than an order of magnitude: runtimes carrying OpenSSL 3.2 or later pay tens of microseconds, runtimes carrying the 3.0 series pay hundreds. The 3.0 series is still the system OpenSSL of long-term-support distributions such as Ubuntu 22.04 and Debian 12, and of any image pinned to an older base layer, which is common in practice (Zerouali et al., 2019).

For software engineering the case matters for three reasons beyond the library itself. First, the cost sits in control flow that no profiler label names: a parse whose failure is caught and discarded appears in profiles as OpenSSL work, and the natural reading of that attribution is “cryptography”. Second, its magnitude is set by a transitive dependency that application developers do not choose directly and rarely record. Third, the obvious fix is unsafe. The resolution block entered the library in version 9.0.0, the release that fixed an RSA-to-HMAC key-confusion vulnerability (Auth0, 2022), and the key-type checks implementing that fix read the type of the key object it produces; removing it the obvious way fails the library’s own key-confusion tests. The code that costs the time is the code that makes a security check work, and the coupling is not visible from the lines being changed. The design that produces the cost (resolving key material on every call, by way of a parse whose failure serves as a type test) can also be named independently of `jsonwebtoken`, and it predicts which of seven libraries pay (Section 7).

Research questions.

RQ1 Where does the per-request cost of token validation actually go?

RQ2 What governs the magnitude of that cost?

RQ3 Does it matter in a running containerised service, and how common is the pattern that avoids it?

RQ4 Is the defect a property of one library, or of a design choice other libraries could share?

RQ5 Can the cost be removed without weakening the library’s key-confusion defence, and who can cause it to be paid?

Contributions. All supported by measurements in the replication package (Katha et al., 2026).

1. A decomposition of the validation call, by two independent methods, locating its dominant component in a discarded key-parse attempt rather than in the signature primitive or in key conversion, and a second decomposition of that attempt into the OpenSSL parse, the V8 exception and the binding between them (Section 4).
2. A causal attribution of its magnitude to the bundled OpenSSL release: the parse reissued from C with no JavaScript engine across 8 identically built OpenSSL releases, a runtime pair that crosses engine and library, and a repetition on a second architecture (Section 5).

- 65 3. A measurement in a containerised service across seven arrival rates, and
66 a survey establishing that the avoiding pattern is rare in three samples
67 of public code (Section 6).
- 68 4. A comparison of seven JWT libraries in four languages showing that
69 the cost follows a design choice (resolving key material on every call,
70 of which a failing asymmetric parse used as a type test is the costliest
71 form) rather than JWT validation as such (Section 7).
- 72 5. An analysis of the constraint on removing that component (Section 8).
73 It is reachable before authentication by any well-formed token; it entered
74 the library with a key-confusion fix and is load-bearing for it, so the
75 natural optimisation fails the library’s key-confusion tests; a narrow fix
76 preserves the defence and makes validation 4.15 times faster, but leaves
77 the parse reachable through an attacker-chosen algorithm header; and
78 only a pre-parsed key removes it for every input.
- 79 6. Guidance for application developers, library authors and performance
80 engineers, derived from the findings and summarised in Table 9 (Sec-
81 tion 9).

82 *Organisation.* Section 2 reviews related work and Section 3 the methodology.
83 Sections 4 to 8 answer RQ1 to RQ5 in turn. Section 9 draws implications
84 for practice, Section 10 discusses threats to validity and future work, and
85 Section 11 concludes.

86 2. Background and related work

87 *What a validation call does.* A JSON Web Token carries a header, a payload
88 and a signature over the first two (Jones et al., 2015). Validating one means
89 decoding two segments, resolving key material into a form the cryptographic
90 library accepts, and verifying the signature. The middle step, key resolution,
91 is the subject of this paper, and it is the step most often left implicit when
92 validation cost is discussed.

93 *Where the cost is usually attributed.* Measurement of layered enforcement in
94 microservices concentrates on transport and proxy layers; Zhu et al. (2023)
95 show that service-mesh sidecar overhead depends on configuration and work-
96 load rather than being a fixed tax. Work on cryptographic library performance
97 has largely stayed at the level of the primitive or the protocol exchange (Naser
98 et al., 2020; Sikeridis et al., 2020). Our result concerns neither: it is the cost

99 of getting key material *into* the library, which we could not find measured
100 directly.

101 *Performance bugs.* Jin et al. (2012) found that most real-world performance
102 bugs follow a few recurring patterns, among them skippable function calls,
103 and that fixes are usually local; Zaman et al. (2012) and Nistor et al. (2013a)
104 show that they are hard to reproduce and are mostly found by code reasoning.
105 In JavaScript, Selakovic and Pradel (2016) identify inefficient API usage as a
106 leading root cause, and their later profiler (Selakovic et al., 2017) targets the
107 *order* in which a program evaluates conditions, the axis on which this defect
108 lives. The defect here is a skippable call, but it has none of the structural
109 signatures detectors look for (Nistor et al., 2013b; Song and Lu, 2014): it is
110 one call per request whose failure is its only observable effect. Because Node.js
111 runs handlers on one event loop, a synchronous per-request cost delays every
112 other client (Davis et al., 2018; Staicu and Pradel, 2018).

113 *Versions, dependencies and environment.* Performance changes across versions
114 in ways maintainers do not anticipate (Kaltenecker et al., 2023; Chen and
115 Shang, 2017), and microbenchmark suites often fail to detect it (Laaber
116 and Leitner, 2018). Container images pin a dependency set implicitly (Cito
117 et al., 2017), routinely ship outdated packages (Zerouali et al., 2019) and
118 do not guarantee reproducible builds (Malka et al., 2026). For OpenSSL 3,
119 maintainers of a major downstream library have stated that parsing and
120 key loading regressed against 1.1.1 (Gaynor and Kehrner, 2026); Section 5.2
121 supplies a controlled measurement for the one path this paper concerns.

122 *Measurement protocol.* Our protocol follows established guidance for managed-
123 runtime benchmarking (Georges et al., 2007; Kalibera and Jones, 2013; Barrett
124 et al., 2017; Laaber et al., 2019). The attribution this paper overturns has
125 the structure Mytkowicz et al. (2009) warn about: a second variable, the
126 bundled OpenSSL version, moving unrecorded alongside the one under study.

127 *Cryptographic APIs and JWT libraries.* Most cryptographic vulnerabilities
128 are misuses of libraries by applications (Lazar et al., 2014; Egele et al., 2013),
129 largely because the interfaces are hard to use (Nadi et al., 2016; Acar et al.,
130 2017); detection has been automated by specifying correct usage (Krüger et al.,
131 2018; Rahaman et al., 2019). Our finding differs: the interface admits a use
132 that is *correct* and costly only for a reason invisible at the call site. Security
133 analyses of JWT implementations consistently find key-handling defects, key

134 confusion among them (Detering et al., 2017; Xu et al., 2023; Yang et al.,
135 2026), and RFC 8725 (Sheffer et al., 2020) codifies the resulting practice.
136 The library studied here has been hardened against key confusion twice:
137 CVE-2015-9235, fixed in 4.2.2 (GitHub Advisory Database, 2015), and CVE-
138 2022-23541 with two related defects, fixed in 9.0.0 (Auth0, 2022). We treat the
139 same surface as a cost site and show the two views are coupled. We also draw
140 on guidance for mining public repositories (Kalliamvakou et al., 2014) and on
141 work showing that exceptions are costly in managed runtimes (Ogasawara
142 et al., 2006) and that discarding handlers are common (Cabral and Marques,
143 2007; Kery et al., 2016).

144 2.1. What we searched for and did not find

145 Table 1 summarises the positioning. Each cell records only what this
146 section states about the work cited; it is a map of the lines of work this paper
147 connects, not an assessment of them.

148 We found no peer-reviewed study of performance regressions across the
149 OpenSSL 3.x series, none tracing the chain from base image to bundled library
150 to application performance, and none treating a correct-but-costly use of a
151 cryptographic interface as a performance defect. We state each as an absence
152 we searched for (details in Supplementary Section S1).

153 3. Methodology

154 Figure 1 gives the study’s workflow: the instruments used for each research
155 question and the tables and figures each produces. Every instrument writes
156 to committed run files, from which every number in the paper is generated
157 (Data availability).

158 3.1. Testbed

159 Unless stated otherwise, measurements were taken on one Apple-silicon
160 machine running macOS 26.5.2 (arm64), referred to throughout as *the*
161 *host*. The host microbenchmarks of Section 4 ran directly on macOS un-
162 der Node.js v26.6.0. Container measurements (Sections 5 and 6) ran in
163 the Linux/arm64 virtual machine of Docker Desktop’s built-in single-node
164 Kubernetes cluster (v1.32.2).

165 The service studied in Section 6 is a Node.js microservice (Express 5.1.0,
166 jsonwebtoken 9.0.3) built on the `node:18-alpine` image, run as a single
167 replica with an Istio sidecar (`istio/proxyv2`; the proxy version was not

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API security*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version as cause*: attributes a cost or defect to the version of a bundled dependency. *Cost-defence coupling*: shows that removing a cost would weaken a security defence. ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API security	Cost	Version as cause	Cost-defence coupling
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	–	–	–
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	–	✓	–	–	–
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	–	–	✓	–	–
Zhu et al. (2023)	–	–	✓	–	–
Jin et al. (2012); Selakovic and Pradel (2016)	–	–	✓	–	–
Kaltenecker et al. (2023); Chen and Shang (2017)	–	–	✓	◦ ^a	–
Zerouali et al. (2019); Cito et al. (2017)	–	–	–	◦ ^b	–
This paper	✓	✓	✓	✓	✓

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured.

168 recorded, and its image digest is in the replication package), and reached
169 over TLS through an NGINX ingress controller that is itself injected into
170 the mesh. It verifies HS256 tokens with a shared secret supplied as a string.
171 Load was generated in open loop by k6 v2.2.0 on the same machine, so
172 the load generator, the ingress, the sidecars and the service share one host
173 (Supplementary Section S2). This is a controlled single-node testbed, not
174 a production deployment; we call it the *containerised service*. The x86_64
175 measurements used the separate machine described in Section 3.8.

176 3.2. Unit of measurement

177 The unit is one operating-system process, not one call. Each process
178 performs 40000 timed iterations of a single condition after a warmup of half

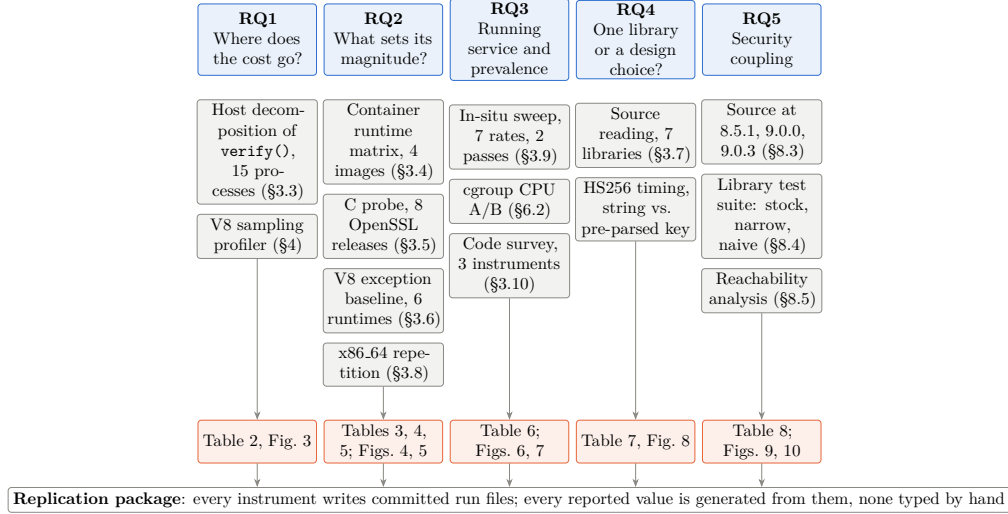


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded attempt for RQ1 (Table 3), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

179 that number, and contributes its median. Reported figures are medians of
 180 per-process medians, with percentile bootstrap intervals taken over processes.

181 Conditions are interleaved within each round rather than run in blocks,
 182 the clock’s own cost is measured and reported rather than subtracted, and
 183 differences between conditions are reported as the median of per-round *paired*
 184 differences, not as the difference of two medians. The process counts per
 185 experiment, coefficients of variation and the reasoning behind these choices
 186 are given in Supplementary Section S2.

187 3.3. Decomposing the call

188 `jwt.verify()` is decomposed into the operations it performs, each timed
 189 in isolation under the same protocol: structural decode; the HMAC primitive
 190 with a string key and with a pre-parsed key; the symmetric key constructor;
 191 and the asymmetric key parse the library attempts first, in both the case
 192 where it throws and the case where it succeeds.

193 The library measured throughout is `jsonwebtoken` version 9.0.3, cloned
 194 at tag v9.0.3. The code path at issue is shown in Figure 2.

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
secretOrPublicKey === 'string' ? Buffer.from(
secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JwtError('secretOrPublicKey is not
valid key material'))
128     }
129   }
130 }

```

Figure 2: Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

195 3.4. Separating the runtime from the deployment

196 A host microbenchmark compared against an in-container measurement
197 cannot support an attribution to deployment: those two environments differ
198 in more than deployment, and any of the differences could carry the effect.
199 Here containerisation is held constant and the runtime is varied instead: four
200 container images spanning two OpenSSL series and three V8 major versions,
201 on one machine under one container engine.

202 The pair carrying most of the inference is Node 18 and Node 20: they ship
203 V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock
204 images Node and OpenSSL versions otherwise co-vary, which would leave the
205 attribution underdetermined. Because that pairing carries the attribution,
206 the runtime identity of each image (Node, OpenSSL and V8 version, libc,
207 architecture and image digest) is recorded as a datum and cross-checked
208 against the versions each measurement row carried. That check found 0
209 mismatches across the matrix. Section 5.1 states what this design alone
210 cannot separate, and Sections 3.5 and 3.6 describe the two further designs

211 that do.

212 3.5. Taking the JavaScript engine out

213 To test the attribution with no engine present, we read the Node source
214 at the two release lines measured and reproduced in C, call for call, what
215 `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three
216 PEM scans for a public key, an RSA public key and a certificate; then
217 `PEM_read_bio_PrivateKey`; then the C side of Node’s error path, which
218 drains the OpenSSL error queue into strings. The program was compiled
219 once per release with one compiler (GCC 13.3.0) and one set of flags, against
220 8 OpenSSL releases from 3.0.16 to 3.6.1, each built from source with identical
221 configuration and linked by run path; each binary reports the version it
222 loaded, and there were 0 mismatches. The protocol is that of Section 3.2, at
223 15 processes of 5000 calls per release and condition. We also time the parts
224 separately and a *successful* parse of an RSA-2048 public key.

225 3.6. Separating the exception from the parse

226 Inside Node the failed parse is followed by a V8 exception that the caller
227 discards. To measure that exception without the parse, a JavaScript bench-
228 mark builds an `Error` of the same message, properties and code (captured
229 from a real failure on the runtime measured) and throws it from 8 frames
230 below the catch, with and without stack capture. It runs under six official
231 Node runtimes (18.20.8, 20.20.2, 22.23.3, 23.11.1, 24.21.0 and 26.10.0) at 15
232 processes of 10000 calls per condition, alongside the real probe. Combined
233 with Section 3.5 this gives a three-way decomposition per runtime: the C
234 cost of the bundled OpenSSL release, the V8 exception, and a remainder
235 attributed to the binding between them.

236 3.7. Other libraries

237 To ask whether the defect is one library’s or a design choice’s, we read the
238 key-handling code of six further libraries, chosen as the most used in each
239 ecosystem: `fast-jwt` and `jose` (JavaScript), `PyJWT` (Python), `golang-jwt`
240 (Go), and `nimbus-jose-jwt` and `jjwt` (Java). For each we answer one
241 question from source: given a string HMAC secret, does it attempt to parse
242 it as an asymmetric key? We then time HS256 verification in each with the
243 secret in two forms, under the same one-condition-per-process protocol at 15
244 processes of 20000 calls. “String” is the form each library’s documentation
245 shows for an HMAC secret, used the way its interface is meant to be used;

246 “pre-parsed” is the library’s own key object, built once. Where a library
247 does not accept a string we use its rawest accepted form and say so. The
248 JavaScript libraries are run under two runtimes, one bundling OpenSSL 3.0
249 and one bundling 3.5, because the `jsonwebtoken` penalty depends on that.
250 The penalty is the median of per-round paired differences between the two
251 forms.

252 3.8. A second architecture

253 Tables 2 and 4 were measured on one ARM64 host. Both were repeated
254 with unchanged scripts on an x86_64 machine (Intel(R) Xeon(R) Processor @
255 2.80GHz, four virtual CPUs), including the four container images of Table 4;
256 Sections 3.5–3.7 were measured on the same machine. That machine is a
257 virtual machine on shared cloud infrastructure, not the dedicated host a
258 replication of magnitudes would require (Laaber et al., 2019; Leitner and Cito,
259 2016). We use it for what it can support: whether the *ordering* of conditions
260 and runtimes holds on a second instruction set, and ratios between conditions
261 measured interleaved in the same rounds. We do not compare its absolute
262 magnitudes with the ARM64 host’s.

263 3.9. In-situ measurement

264 The containerised service exposes a histogram timing the verification call
265 alone. The cumulative histogram is read before and after each arrival-rate
266 step and the readings are differenced; means are the difference of `_sum` over
267 the difference of `_count`, so they do not depend on bucket boundaries. Seven
268 arrival rates are driven, and the sweep is run twice on one process, ascending
269 and then descending. We report both passes; their range is two observations,
270 not a confidence interval. This establishes that the cost is present in the
271 service and of what order, not the shape of the cost-versus-rate relation. The
272 magnitude is checked separately by cgroup CPU accounting (Section 6.2).

273 3.10. Prevalence: three instruments, three frames

274 Prevalence is approached with three instruments that fail in different ways.
275 Their sampling frames differ and are not interchangeable, so no population
276 proportion is computed from them. All three were captured on 2026-09-18.

277 The three are population co-occurrence counts from code search (two
278 languages by two import forms), a classified convenience sample of call sites
279 with the key argument traced within its file, and a targeted counter-search
280 that fetches every file co-occurring the library with the pre-parsing interface

281 and adjudicates it by hand, deliberately biased toward finding the pattern
 282 (Supplementary Section S2 details each frame).

283 No population proportion is computed: the co-occurrence signal carries a
 284 false-positive rate (5 of 13 automatically flagged call sites on hand inspection),
 285 the sample has no inclusion probabilities, and the counter-search is selected
 286 on the outcome. Results are therefore reported *in our sample* throughout.

287 *Host state.* Because the service shares its host with its load generator, ingress
 288 and sidecars, every in-situ window carries the host load average observed
 289 during it, and this paper makes no capacity claim (Supplementary Section S2).

290 4. RQ1: Where the cost goes

291 4.1. The decomposition

292 Table 2 gives each operation in the validation path, timed in isolation on
 293 the host under the protocol of Section 3, and Figure 3 plots it.

Table 2: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.2).

Operation	Median (μ s)	95% interval (μ s)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

294 The two emphasised rows are the result. The signature primitive costs
 295 2.208 μ s of a 29.92 μ s call. The conversion to which the remainder is most
 296 readily attributed costs 0.583 μ s. The discarded attempt that precedes that
 297 conversion costs 18.04 μ s, which is 60% of the whole call and its largest single
 298 component.

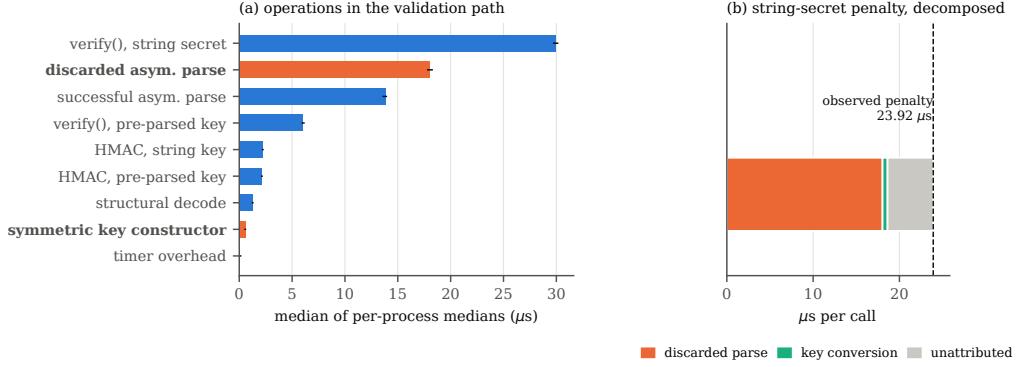


Figure 3: (a) Every operation in Table 2, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.3).

299 Supplying a pre-parsed key instead of a string reduces the call by 23.92 μs
300 [23.58, 24.00] over 15 paired rounds.

301 *Key conversion does not explain it.* Once the signature primitive is shown
302 to be small, one explanation for the remainder suggests itself immediately:
303 that the cost lies in converting key material from a string into a key object
304 on every call. It is the natural explanation, and Table 2 refutes it. That
305 conversion costs 0.583 μs , so the attribution is wrong by roughly the ratio
306 between that figure and 18.04 μs .

307 A second observation is usually offered in support of the conversion account:
308 an asymmetry between HMAC and RSA costs, read as conversion being dearer
309 than parsing a PEM. It is backwards. A successful asymmetric parse costs
310 13.83 μs against 0.583 μs for the conversion. The real asymmetry is that the
311 RSA path’s parse *succeeds*, making its cost useful work, while the HMAC
312 path’s fails, making it waste.

313 4.2. What the 18.04 μs is made of

314 The 18.04 μs is the cost of the discarded attempt taken whole. It contains
315 at least two things: the work OpenSSL performs before concluding that the
316 input is not an asymmetric key, and the construction and unwinding of the
317 exception that returns that conclusion to the library. Table 2 already bounds

Table 3: The discarded attempt decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

the split: a *successful* asymmetric parse (the same entry point, reaching a conclusion and throwing nothing) costs 13.83 μs , the same order as the failing one, so the exception cannot be most of it. Table 3 replaces the bound with a decomposition, combining the C measurement of the same OpenSSL release (Section 3.5) with the exception baseline (Section 3.6) on six official runtimes.

Three things follow. First, on every runtime bundling OpenSSL 3.0 the C parse alone matches the probe to within a few percent: 101% of it under Node 18.20.8 and 103% under Node 23.11.1, the excess over 100% being the difference between our build of the release and Node’s. There the discarded attempt *is* the OpenSSL parse, and it would cost the same with no JavaScript engine present. Second, the exception is small and nearly constant across runtimes: the Node-shaped error costs 4.781 μs under Node 18.20.8 and 3.421 μs under Node 26.10.0 (1.2% and 19.0% of the respective probes), and roughly half of it is stack capture (with capture disabled, 2.204 and 1.930 μs). Disabling stack capture for the real probe saves 5.397 μs and 1.767 μs on the same two runtimes, the same order as the synthetic figure. Third, on runtimes bundling OpenSSL 3.5, where the parse is cheap, the three parts are of comparable size: under Node 26.10.0 the parse is 47% of the probe, the exception 19.0% and the binding 34.1%.

The component is therefore a discarded *parse attempt*, not an exception

cost. Where it is expensive the exception is about a hundredth of it, and even where it is cheap the exception is a fifth. These runtimes are Linux builds on x86_64; the ARM64 host figure of Table 2 was not itself decomposed this way, and its 18.04 μ s is of the same order as the OpenSSL 3.5 rows here, not the OpenSSL 3.0 rows.

4.3. Checks on the decomposition

The discarded parse and the conversion together account for 77.9% of the 23.92 μ s string penalty; the 5.292 μ s residual [4.916, 5.501] lies in the library’s own type and claim checking and is reported as unattributed rather than absorbed into the headline. Because the decomposition relies on differencing, it was also checked against V8’s sampling profiler, which needs no arithmetic on our part: the asymmetric key parse accounts for a median 66.2% of self time on the host, 95.3% under Node 18 and 61.5% under Node 26, and does not appear among the top frames of the pre-parsed path. Two methods, one answer (Supplementary Section S3).

5. RQ2: What governs the magnitude

Table 4 varies the runtime with containerisation held constant, and Figure 4 plots the two columns that matter.

Table 4: The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. [‡]The host row is from Table 2 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10).

Runtime	OpenSSL	V8	Disc. parse (μ s)	Pre-parsed (μ s)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host [‡]	3.6.3 [†]	14	18.04	6.042

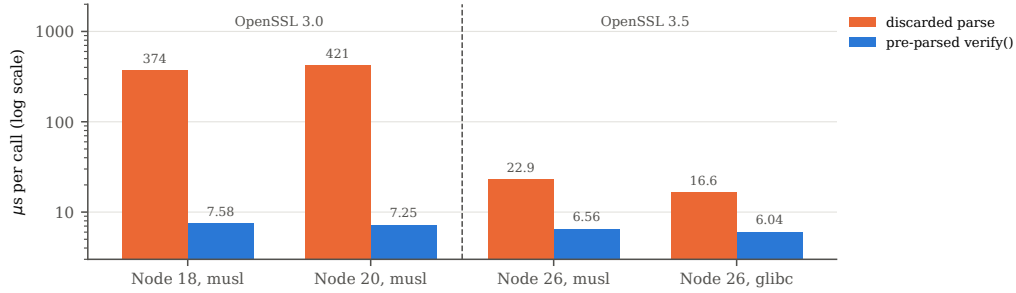


Figure 4: The four container rows of Table 4 on a logarithmic axis. The discarded parse spans $25.36\times$ across images while the pre-parsed call spans $1.26\times$. The dashed rule marks the OpenSSL series boundary. Within this matrix it is not a controlled boundary, because the two fast images also carry a newer engine and a different build (Section 5.1); Section 5.2 controls it. The uncontainerised host row is omitted because its comparison with the container rows is confounded.

5.1. What the matrix shows, and what it cannot separate

Node 18 and Node 20 ship different major versions of V8 (10 and 11) and share the OpenSSL 3.0 series; both pay of order $373.79\text{--}420.62\ \mu\text{s}$, so within that series the engine does not move the quantity. The effect is confined to key parsing: across the four container rows pre-parsed validation spans $1.26\times$ and the HMAC primitive $1.39\times$, while the discarded parse spans $25.36\times$ and a successful parse moves with it ($11.74\times$). The matrix cannot, however, separate OpenSSL from the engine across the series boundary, because the fast images also carry a newer engine and a different build; nor does it isolate containerisation (Supplementary Section S4). The causal wording in this paper rests on the crossed designs below, not on the matrix.

5.2. Caused by OpenSSL: the parse without an engine

Table 5 removes Node entirely: the OpenSSL calls `createPublicKey()` makes for an HMAC secret, issued from C against 8 OpenSSL releases built identically, and Figure 5 plots its first and last columns.

In plain C the failure costs $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, a ratio of 46.69 [$46.16, 46.90$]. Nothing else differs between those two measurements: the program, the compiler, the flags, the machine and the protocol are the same. The magnitude is therefore *caused* by the OpenSSL release. That is a claim about this code path in these releases; it says nothing about OpenSSL performance generally.

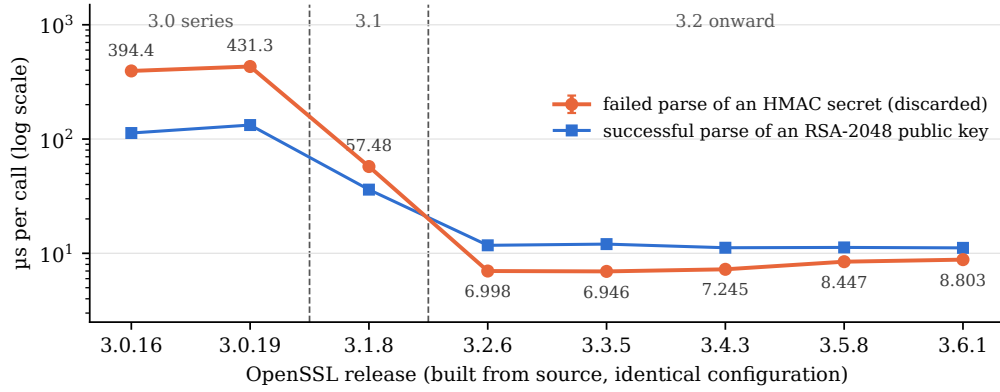


Figure 5: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration (Table 5, column *Full*), beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

Table 5: The failed asymmetric parse issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one compiler and one set of flags (x86_64), in μ s. Medians of per-process medians over 15 processes of 5000 calls. *Full*: everything `createPublicKey()` asks of OpenSSL for an HMAC secret, including the error-string drain; 95% intervals for it are in the replication package and are within 0.9% of the median for every release. Its parts follow. *Successful parse*: an RSA-2048 public key through the same path, for comparison.

OpenSSL	Full	Private-key parse	Public-key scans	Error strings	Successful parse
3.0.16	394.4	390.9	2.287	0.796	112.9
3.0.19	431.3	424.0	2.231	0.869	132.6
3.1.8	57.48	54.84	2.196	0.497	36.09
3.2.6	6.998	4.487	2.057	0.459	11.75
3.3.5	6.946	4.465	2.074	0.474	12.04
3.4.3	7.245	4.554	2.098	0.500	11.19
3.5.8	8.447	5.757	2.051	0.483	11.26
3.6.1	8.803	5.985	2.234	0.472	11.16

377 The table also says where in OpenSSL the cost lives and when it went
378 away. Almost all of it is the private-key parse (99% of the full sequence on

379 3.0.16), the step at which OpenSSL 3 hands the input to its provider-based
380 decoders; the three public-key scans reject the input at the PEM header and
381 are cheap in every release, and the error strings cost under a microsecond.
382 The fall came in two steps: to 57.48 μ s in 3.1.8, and to 6.998 μ s in 3.2.6, after
383 which it is flat to within a few microseconds through 3.6.1. We measured the
384 steps, not the source changes that produced them; identifying those would
385 take a bisection of the OpenSSL history that we have not done. A successful
386 parse of real key material fell with it, which is why the effect is confined to
387 key parsing in general rather than to the failure path.

388 *Engine and library crossed.* The official runtimes of Section 3.6 contain the
389 pairing stock container images lack: Node 22.23.3 ships V8 12.4.254.21 with
390 OpenSSL 3.5.8, while Node 23.11.1 ships the *newer* V8 12.9.202.28 with the
391 *older* OpenSSL 3.0.16. The probe costs 18.98 μ s on the first and 383.5 μ s
392 on the second. The Node 18/20 pair of Table 4 showed that a change of
393 engine at a fixed OpenSSL series leaves the cost where it was; this pair shows
394 that a newer engine does not rescue an older OpenSSL. Together with the C
395 measurement, the engine is excluded from both directions.

396 5.3. A second architecture

397 The runtime matrix and the host decomposition were repeated with the
398 same images and scripts on x86_64 (Section 3.8). The property the attribution
399 rests on, every OpenSSL 3.0 image slower than every OpenSSL 3.5 image,
400 holds there too: the discarded parse spans 30.50 $\times$ across the images while
401 pre-parsed validation spans 1.31 $\times$. The host decomposition keeps its ordering:
402 the discarded parse is 19.42 μ s of a 36.48 μ s call (53%). The full table is in
403 Supplementary Section S4.

404 6. RQ3: The containerised service, and prevalence

405 6.1. In-situ cost across arrival rates

406 The containerised service was driven at seven arrival rates, each measured
407 by the paired-scrape difference of Section 3.9, with the whole sweep run
408 ascending and then descending on one process. Figure 6 plots both passes at
409 each rate for the configuration the service deploys, a string secret (the values
410 are tabulated in Supplementary Section S5).

411 One observation follows, and it is the one this section claims: the cost is
412 present in the containerised service and is far larger than the isolated host

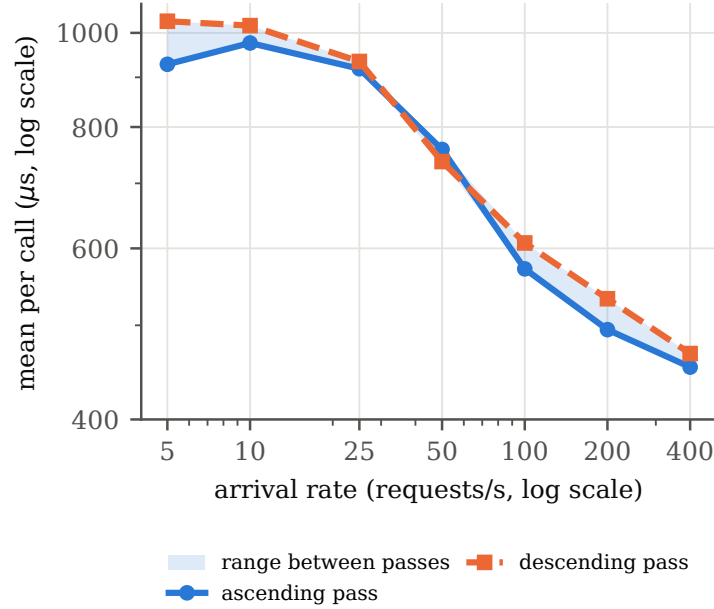


Figure 6: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged: Supplementary Section S5 reports that they differ systematically, and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

413 figure of Table 2, consistent with Section 5 in that the service’s runtime is
 414 the one carrying the older OpenSSL.

415 Two things in these measurements need care, and we set them out explic-
 416 itly.

417 *Wall-clock exceeds the prediction; processor time does not.* The service run-
 418 time’s discarded parse costs 373.79 μ s (Table 4), and the remainder of the
 419 validation path is of order 7.58 μ s, giving an expected total near 381 μ s. The
 420 wall-clock means observed across the sweep are 453–1028 μ s, exceeding that
 421 by 71–647 μ s. Contention inflates a duration rather than deflating it, so these
 422 are upper bounds, and Section 6.2 shows where the excess is not: measured
 423 as processor time, the validation call costs within 0.8% of the prediction. The
 424 excess is therefore time the call spent off the processor on a shared host, not
 425 additional work. We did not decompose that off-processor time further, and
 426 no claim below depends on it.

427 *Neither the decline with rate nor the gap between passes is interpreted.* Cost
 428 falls from 928.24 μs at 5 requests/s to 452.82 μs at 400 requests/s, and the
 429 two passes differ systematically rather than randomly. Several mechanisms
 430 unrelated to validation could produce either, we did not run the conditions that
 431 would distinguish them, and no claim here depends on them (Supplementary
 432 Section S5).

433 6.2. Corroboration: the cost is processor time, and it matches the prediction

434 An in-situ figure of this magnitude has two readings. Either it is work the
 435 service performs, or it is time the request spends waiting, queued behind other
 436 requests, descheduled by a contended host, or inflated by the instrument.
 437 We separate them with an A/B of the same service at one arrival rate, with
 438 validation enabled in one window and disabled in the next, on the same
 439 pod running the same image, and read the processor time the application
 440 container was charged from its cgroup accounting rather than from a clock.
 441 Table 6 reports it.

Table 6: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

442 Enabling validation adds 384.6 μs of processor time per call. The runtime
 443 matrix predicts 381 μs for the same work on the same runtime (Section 6.1);
 444 the two agree to within 0.8%. They are obtained by instruments that share
 445 no code: cgroup CPU accounting on the service’s pod, and a single-process
 446 microbenchmark in an isolated container. A request that queues or is de-

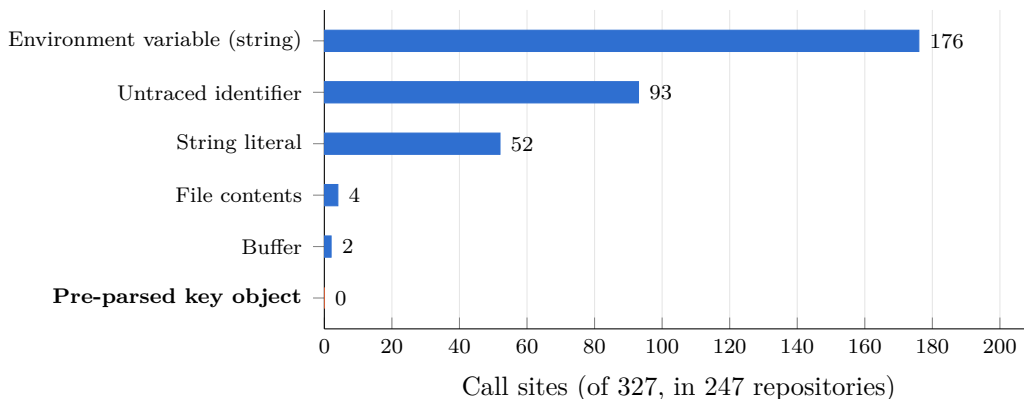


Figure 7: Where the secret passed to the validator comes from, for the 327 call sites classified in our sample. No call site passes a pre-parsed key object, the one form that avoids the discarded parse for every input. Independently, 309 of the 327 call sites (94%) pass no algorithm restriction. Counts describe the sample, not the population (Section 3.10).

447 scheduled accrues wall-clock time but no CPU, so this figure cannot contain
 448 queueing delay.

449 Three observations weigh against reading the difference as an artefact of
 450 the two windows. The sidecar, in the same pod under the same contention, did
 451 not rise with the application: 53.40 millicores disabled against 49.13 enabled.
 452 The one-second CPU samples of the two arms do not overlap at all (disabled
 453 never exceeded 100.577 millicores in 127 samples, enabled never fell below
 454 114.963 in 127), which separates the arms without a distributional assumption.
 455 And event-loop lag is unchanged, so neither arm was near saturation. The
 456 two windows are two observations, not a sample: the difference carries no
 457 interval and we compute none.

458 6.3. How common is the avoiding pattern, in our sample?

459 Three instruments, three frames, reported separately as Section 3.10
 460 requires. All counts were captured on 2026-09-18. None of the figures below
 461 is a population proportion.

462 *Population co-occurrence.* In code-search counts over public repositories, no
 463 cell exceeds 0.0303% of files importing the library that also mention the
 464 pre-parsing interface (Supplementary Section S5).

465 *Classified call sites.* Of 327 call sites classified across 247 distinct repositories
466 in our sample, 0 pass a pre-parsed key. Environment variables account for
467 176 of those call sites, untraced identifiers and member expressions 93, string
468 literals 52, file contents 4 and buffers 2 (Figure 7). No file in this corpus
469 contains the pre-parsing interface at all, in any spelling, so the zero reflects
470 the pattern’s absence from the sample rather than a classifier that cannot
471 see it. Incidentally, 309 of 327 call sites (94%) pass no algorithm restriction,
472 which Section 8.4 returns to.

473 *Targeted counter-search.* A sample can only show a pattern is rare if it
474 could have found it. Every file co-occurring the library with the pre-parsing
475 interface was therefore fetched: 191 files, of which 120 contain a validation
476 call, spanning 112 distinct repositories. An automated pass flagged 13 call
477 sites as passing a pre-parsed key and all were then read by hand: 5 are false
478 positives (projects defining their own helper function of the same name, one
479 of which returns a string), 1 could not be resolved within its file, and 7 are
480 genuine, across 6 distinct projects.

481 *The rate, with its denominator.* Among the 112 repositories that both refer-
482 ence the pre-parsing interface and call the validator, 6 use it: 5.36%. That
483 denominator is the most favourable available, since those projects demonstra-
484 bly know the interface exists, and it is not the population. Within this frame
485 the pattern is rare rather than absent, and it is rare even among projects
486 already aware of it. We do not extrapolate beyond the frame.

487 7. RQ4: One library, or a design choice?

488 A cost found in one library is an observation about that library. It becomes
489 a statement about a class of defect only if the mechanism can be named
490 independently of the library and predicts which other libraries have it. The
491 mechanism here can be named: *resolving key material of unknown type by*
492 *attempting the asymmetric parse first, and treating its failure as the signal*
493 *that the material is a shared secret.* Table 7 tests the prediction against six
494 further libraries.

495 *What the source says.* Only `jsonwebtoken` attempts the asymmetric parse.
496 The others resolve the ambiguity in one of three ways, none of which involves
497 a parse that can fail on a shared secret. *Inspect, then dispatch:* `fast-jwt`
498 examines the text of the key (whether it carries a PEM header or is a JSON

Table 7: HS256 verification across seven JWT libraries in four languages (x86_64), in μs . *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.7). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

499 Web Key) and calls exactly one constructor, skipping even that inspection
500 when every permitted algorithm is HMAC; PyJWT likewise inspects the text,
501 and rejects PEM- or SSH-looking material offered as an HMAC key rather
502 than trying to parse it. *Refuse the ambiguous input*: `jose` does not accept
503 a string for an HMAC key, and `golang-jwt` accepts only a byte slice for
504 HMAC verification, returning a key-type error for anything else. *Type the*
505 *key*: `jjwt` requires a `SecretKey` for HMAC and a `PublicKey` for signatures,
506 so the distinction is made by the Java type system; `nimbus-jose-jwt` offers
507 a constructor that takes a string and converts its bytes without parsing them.
508 Refusing a string does not by itself remove per-call work, however: `jose`,
509 handed raw bytes, imports them into a `WebCrypto` key on every call.

510 *What the measurements say*. Of the seven libraries, two pay for being handed
511 their raw form rather than a key object, in the sense that the paired interval
512 on the penalty excludes zero; for the other five it does not, and their point
513 estimates are within a microsecond of zero. The two are `jsonwebtoken`
514 and `jose`, and they pay for different reasons. `jsonwebtoken` performs the
515 failing asymmetric parse: its penalty is 465.9 μs under OpenSSL 3.0.19 and
516 24.56 μs under OpenSSL 3.5.8, a string-to-pre-parsed ratio of 46.55 and 3.48
517 respectively (Figure 8). `jose` parses nothing that can fail, but imports the
518 raw bytes into a `WebCrypto` key on every call: 42.66 μs and 24.80 μs under

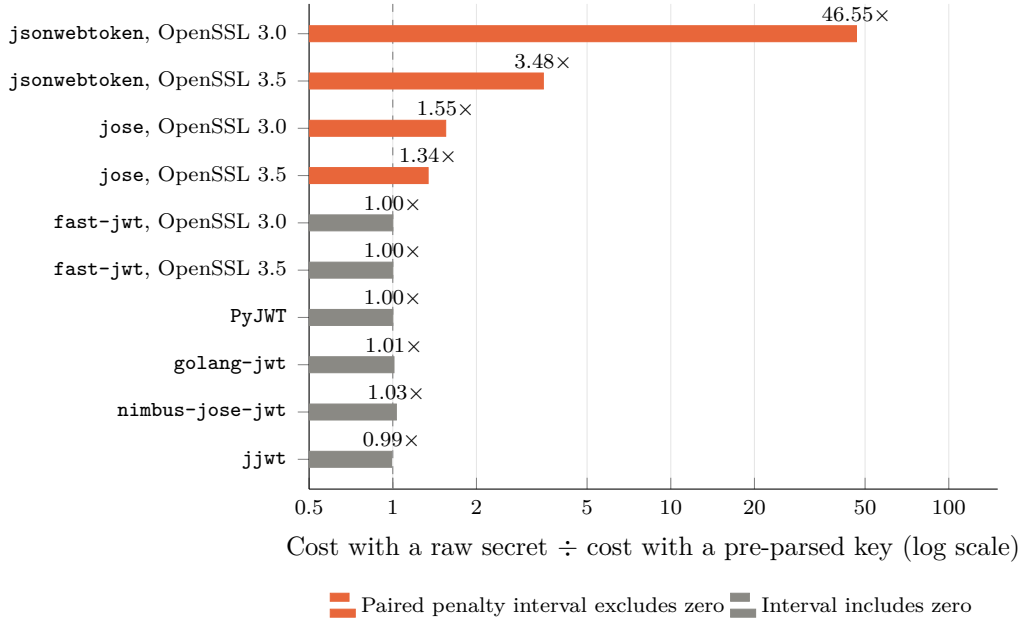


Figure 8: How much each library pays for being handed its raw secret rather than its own pre-parsed key object: the ratio of the two costs in Table 7, on a log scale. A ratio of one (dashed line) means no per-call penalty. Orange: the paired 95% interval on the penalty excludes zero. Only `jsonwebtoken` performs the failing asymmetric parse, and only it pays by more than a small factor; `jose` pays for re-importing raw bytes on every call.

519 the same two runtimes. Both penalties grow on OpenSSL 3.0, so the OpenSSL
 520 release sets the price of per-call key handling generally; but the failing parse
 521 is the costliest form of it, about 11 times `jose`'s on OpenSSL 3.0. Two of the
 522 libraries that pay nothing resolve the key once, when a verifier or parser is
 523 built (`fast-jwt`, `jjwt`), so a raw secret costs nothing per call by construction;
 524 that is itself the design point.

525 *What this establishes.* The libraries without the problem matter as much as
 526 the ones with it. All seven validate the same kind of token with the same
 527 primitive (against the same OpenSSL, in the JavaScript cases) and five pay
 528 nothing for a raw secret. The cost therefore follows a design choice, resolving
 529 key material on every call, and not JSON Web Token validation, HMAC or
 530 OpenSSL as such: OpenSSL 3.0 sets the price, but only a library that makes
 531 that choice is charged it, and one that makes it by way of a failing parse is

532 charged the most. The failing parse is exactly what the order-of-evaluations
533 profiler of Selakovic et al. (2017) is designed to find, and the alternatives
534 above are the fixes that line of work suggests: reorder the test, make it cheap,
535 or move it off the per-call path. The comparison does not estimate how many
536 libraries make the choice; it shows that making it is what matters.

537 8. RQ5: Removing the cost safely

538 Sections 4–7 treat the discarded attempt as a cost. This section asks
539 who can cause it, why it is there, and what removing it does to the library’s
540 defences. Its evidence is the library’s source at the releases named, the
541 library’s own test suite, and the measurements already reported; it introduces
542 no new timing condition, and where a statement rests on reading code rather
543 than on measurement we say so.

544 8.1. *Threat model*

545 The adversary is a remote client that can place arbitrary bytes where the
546 service expects a bearer token, including every field of the token header, and
547 knows which library the service uses. It does not know the shared secret
548 and cannot produce a valid HMAC. The service verifies HS256 tokens with a
549 shared secret supplied as a string, the form the library’s documentation shows
550 and the form every call site in our sample uses (Section 6.3). The asset of
551 interest is key-type separation: a token declaring an HMAC algorithm must
552 never be verified against asymmetric key material, nor the reverse (Sheffer
553 et al., 2020). Out of scope are compromise of the secret, defects elsewhere
554 in the library, and availability. We did not drive adversarial traffic at the
555 containerised service and make no availability or capacity claim; the question
556 here is which inputs reach the discarded attempt, not how many a service
557 can absorb.

558 8.2. *The attempt precedes authentication*

559 Figure 9 shows the order in which `verify()` performs its checks at
560 tag v9.0.3. Reaching the key-resolution block of Figure 2 requires only
561 that the token have three dot-separated segments, a header and payload that
562 decode as JSON, and a non-empty signature segment (lines 59–118). Nothing
563 on that path depends on the secret. The permitted-algorithm list is consulted
564 afterwards, at line 144; the key-type checks at lines 148–160; the signature at
565 line 165.

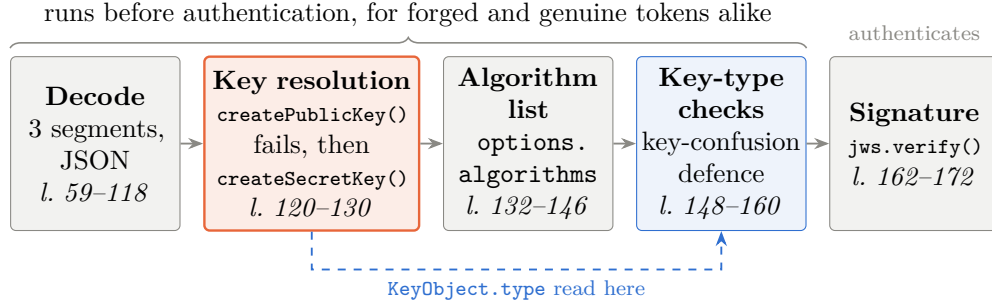


Figure 9: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded attempt sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

Two consequences follow. A token that will be rejected pays the discarded attempt in full: $18.04 \mu\text{s}$ on the host, and $373.79 \mu\text{s}$ on the runtime the studied service deploys (Table 4). And restricting permitted algorithms, the defence RFC 8725 recommends and which 94% of the call sites in our sample omit, does not avoid it, because the restriction is enforced after resolution. The cost is therefore under the control of any client that can reach the endpoint, not only of the service’s legitimate load. That is a statement about reachability, established from the source; its magnitude is the one measured in Sections 4–6.

8.3. The attempt arrived with a key-confusion fix

We compared `verify.js` in the published packages of versions 8.5.1, 9.0.0 and 9.0.3. In 8.5.1 the secret is passed to the signature routine as given, with no key-object resolution. The resolution block of Figure 2 first appears in 9.0.0 and is unchanged, apart from whitespace, through 9.0.3. Version 9.0.0 is the release that fixed CVE-2022-23541 and CVE-2022-23539 (Auth0, 2022), and the checks implementing those fixes (the comparison of the declared algorithm with the key’s type at lines 148–152 and the validation of asymmetric key types at lines 154–160) operate on the key object the resolution block produces. The attempt is thus not an oversight in otherwise untouched code. It is part of a security fix, and its price is set by whichever OpenSSL release a deployment happens to bundle (Section 5).

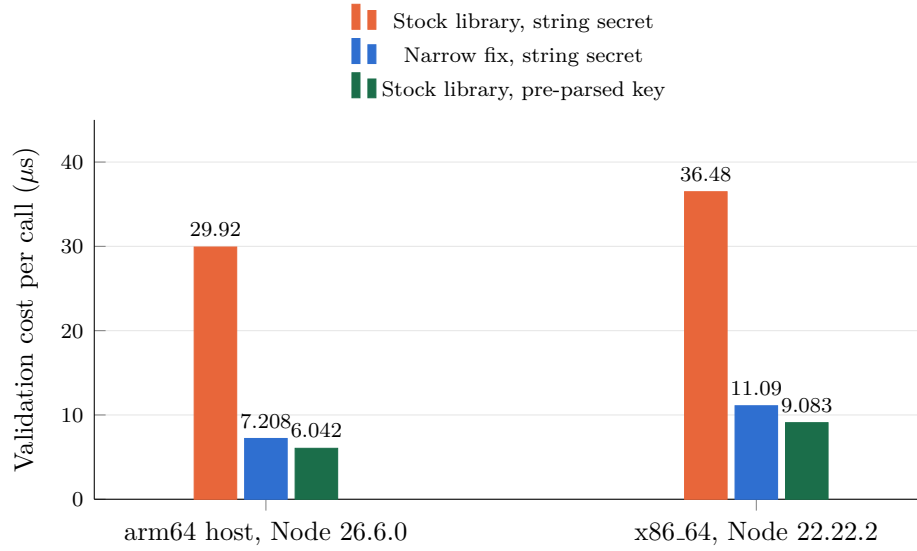


Figure 10: Per-call HS256 validation cost under the stock library with a string secret, under the narrow fix with the same string secret, and under the stock library with a pre-parsed key, on both machines (medians of per-process medians). The narrow fix recovers most of the gap without any change to calling code; only the pre-parsed key removes the attempt for every input (Section 8.5).

586 This is consistent with the earlier upstream report of the symptom (Sec-
587 tion 8.6); we did not time version 8.5.1 ourselves.

588 8.4. A fix, and a constraint on it

589 The fix is to dispatch on the token’s declared algorithm before attempting
590 the asymmetric parse, so that an HMAC token with an ordinary string secret
591 is resolved directly. Applied, this takes the call from 29.92 μs to 7.208 μs , a
592 factor of 4.15, and to within 1.208 μs of supplying a pre-parsed key, with no
593 change in calling code. That residual is the median of the per-round paired
594 differences, not the difference of the two medians; the two are not identical
595 and the former is reported throughout.

596 Figure 10 shows the same comparison on the x86_64 machine of Section 3.8,
597 where the narrow fix takes the call from 36.48 μs to 11.09 μs against 9.083 μs
598 with a pre-parsed key.

599 We tested it against the library’s own suite rather than only against
600 assertions of our own. Cloned at tag v9.0.3, the patch applies cleanly and the
601 suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

602 These figures are from the host. The practical benefit is largest where the
603 discarded parse is largest, of order 373.79 μ s on the runtime the containerised
604 service uses (Table 4), and we did not measure the patched library on the
605 container runtimes. The reported factor of 4.15 is therefore a lower bound
606 on the improvement available in the containerised service’s runtime, and we
607 state it as a host result rather than as a general one.

608 *The constraint, which is the more important half.* The discarded attempt is
609 not purely wasteful. The *success* of that parse on asymmetric key material
610 is what produces a key object whose type a later check depends on, and
611 that check is what prevents symmetric and asymmetric material from being
612 interchanged (Sheffer et al., 2020). A change that dispatches on the declared
613 algorithm while relaxing which key material may take the fast path alters
614 which inputs reach that check.

615 The fix must therefore be narrow: the fast path is restricted to string
616 secrets that cannot be asymmetric key material, so anything that might
617 be resolved exactly as it does today. Removing that material restriction
618 (the form of the optimisation we argue against) produces 2 failures in the
619 library’s own suite, both in its key-confusion tests (“should not allow HMAC
620 verification with an RSA key in PEM format” and, under “signing with pub
621 key as symmetric”, “should not verify”). All of these outcomes (the suite
622 unchanged under the narrow patch, two key-confusion failures without its
623 guard) reproduce on the x86_64 machine of Section 3.8 under a different Node
624 and OpenSSL release (Katha et al., 2026).

625 *The existing suite already guards the property.* No new regression test is
626 needed: the library’s existing suite constrains the change, and a contributor
627 proposing the unrestricted optimisation would be caught by tests they are
628 required to run. The finding is therefore not a missing test but a hidden
629 coupling: the cost documented in Section 4 cannot be removed by the obvious
630 route, and the reason is a property the maintainers already test for but which
631 is not evident from the code being changed.

632 The independent defence against this class of confusion is for the caller to
633 restrict permitted algorithms (Sheffer et al., 2020), and 94% of the call sites
634 surveyed in Section 6.3 do not.

635 8.5. What the narrow fix leaves reachable

636 The narrow fix selects its fast path on the token’s declared algorithm,
637 and the declared algorithm is chosen by whoever constructs the token. For a

Table 8: Which tokens reach the discarded key-parse attempt, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.4. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

638 token declaring an HMAC algorithm and a plain string secret the parse is
639 skipped. For a token declaring any other algorithm the patch falls through
640 to the original resolution, so `createPublicKey()` is attempted on the shared
641 secret and fails exactly as it does today; the token is then rejected by the
642 algorithm list or the key-type check, but only after the parse. Table 8 sets
643 out which inputs reach the attempt under each configuration. The narrow
644 fix removes the cost from legitimate traffic, not from traffic an adversary
645 constructs. Only a pre-parsed key object removes it for every input, because
646 the resolution block is skipped entirely when the configured key is already a
647 `KeyObject` (line 120).

648 This follows from the code and was not separately timed: we did not
649 measure the patched library on tokens declaring a non-HMAC algorithm,
650 and the per-call cost such a token incurs is that of the discarded attempt
651 measured in Sections 4 and 5. A variant that dispatched on the *configured*
652 algorithms rather than the declared one would close the gap for callers who
653 restrict permitted algorithms to HMAC, since tokens declaring anything else
654 are rejected regardless; it is the choice `fast-jwt` makes (Section 7). We did
655 not implement or test it, it would help only the minority of callers who restrict
656 algorithms, and it would have to keep the same guard on key material.

657 8.6. Reported upstream

658 One earlier report of the symptom exists: an issue opened in April 2024¹
659 reports HS256 verification in the 9.x series 34 times slower than in 8.5.1, on a

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

Node release bundling OpenSSL 3.0, with no diagnosed cause. We reported the mechanism, with a minimal reproduction, per-runtime measurements and the OpenSSL finding, as a separate issue², and a pull request implementing the narrow fix with its guard is open against it. At the time of writing neither has received a maintainer response.

9. Implications for practice

Table 9 maps each finding to the evidence behind it and to the stakeholder who can act on it. The paragraphs that follow expand on each recommendation.

For application developers: pre-parse key material once. On the host this removes 23.92 μ s per validation [23.58, 24.00], and far more on a runtime bundling OpenSSL 3.0. It requires no architectural change and no movement of a trust boundary, and it is the only remedy that removes the attempt for every input, including tokens an adversary constructs (Table 8). Callers should also restrict permitted algorithms (Sheffer et al., 2020), which 94% of the call sites we surveyed do not; that is an independent defence, not a substitute, because the restriction is checked after the parse (Section 8.2).

For performance engineers: treat the base image as part of the measurement. The same code, library and architecture differ by 25.36 $\times$ on this path across the container images measured, because they pin different OpenSSL releases. A performance figure quoted without its base image is not reproducible, and a regression after a base-image bump may have nothing to do with the application. Profile before relocating a cost believed to be cryptographic.

For library authors: do not use a failing parse as a type test. Section 7 found three designs that avoid the cost (inspect the material and dispatch, refuse ambiguous input at the interface, or type the key), all shipping in maintained libraries. What they share is that resolution happens once, or not at all, rather than on every call.

For maintainers and reviewers: review performance changes to verification as security changes. The cheapest-looking optimisation of this path fails the library’s key-confusion tests, and the property it breaks is not evident from

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

Table 9: Summary of findings, evidence and implications, by research question and by the stakeholder who can act on each.

RQ	Finding	Evidence	Implication (who acts)
RQ1	The signature is 2.208 μ s of a 29.92 μ s call; a discarded key-parse attempt is 18.04 μ s (60%).	Table 2, Fig. 3; profiler	Profile before relocating a “cryptographic” cost; attribution by intuition was wrong twice here (<i>performance engineers</i>).
RQ2	The parse costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, issued from C.	Tables 4, 5; Fig. 5	Record the base image and bundled OpenSSL with every performance figure; treat base-image bumps as performance changes (<i>developers, DevOps</i>).
RQ3	In a containerised service validation adds 384.6 μ s of CPU per request; no sampled call site pre-parses its key.	Table 6; Figs. 6, 7	Pre-parse key material once at startup: a one-line change (<i>application developers</i>).
RQ4	Of seven libraries in four languages only one attempts the parse; five pay nothing for a raw secret.	Table 7, Fig. 8	Do not use a failing parse as a type test; resolve keys once, inspect then dispatch, or type the key (<i>library authors</i>).
RQ5	The parse is load-bearing for a key-confusion defence; the naive optimisation fails 2 library tests, the narrow fix passes all 511 and is 4.15 $\times$ faster.	Table 8; Figs. 9, 10	Review performance changes to verification code as security changes; run the key-confusion tests (<i>maintainers, reviewers</i>).

the lines being changed (Section 8.4). Changes to token verification should be run against the key-confusion tests and timed on the oldest cryptographic library a deployment may bundle.

Relocation is not the first remedy. A cost believed to be cryptographic invites relocation to a sidecar or gateway. The cost measured here is not cryptographic and is largely avoidable in place, so an architectural remedy that also moves a trust boundary should not be the first thing reached for (Supplementary Section S7).

699 10. Threats to validity and future work

700 We summarise the main threats here; Supplementary Section S6 discusses
701 each in full.

702 *Construct validity.* The decomposition relies on differencing, which we cross-
703 check with a sampling profiler (Section 4.3). The C probe reproduces the
704 OpenSSL calls Node makes, as read from the Node source, but not Node’s own
705 build of OpenSSL, so it approximates the parse cost inside a given runtime;
706 the causal claim compares releases under one build configuration and does
707 not inherit that approximation. The exception baseline reconstructs Node’s
708 error from JavaScript rather than from C++. The security analysis rests on
709 reading the source and running the library’s own tests, not on adversarial
710 traffic, so the paper makes no availability claim.

711 *Internal validity.* The in-situ evidence is one process sweeping seven rates
712 twice, and the A/B is two consecutive windows at unequal host load. We
713 therefore draw no inference from the width of in-situ ranges or from wall-
714 clock comparisons between windows; the magnitude claim rests on cgroup
715 CPU accounting, which contention cannot inflate, and it agrees with the
716 microbenchmark prediction. The headline host figure predates the per-
717 measurement OpenSSL field; its version is inferred from the run record,
718 and three measurements that carry their version bound the effect of that gap.

719 *External validity.* The primary measurements come from one ARM64 host;
720 the x86_64 repetitions ran on a shared cloud virtual machine, from which we
721 take only orderings and ratios between interleaved conditions. The service is
722 a controlled single-node testbed, not a production deployment, and a service
723 that caches verified tokens pays the cost once per token rather than once
724 per request. Seven libraries at one version each show that the cost follows a
725 design choice; they do not estimate how often the choice is made.

726 *Reliability of the survey.* The three prevalence instruments have different
727 frames and none supports a population proportion. Call sites are resolved
728 within one file only, and the flagged call sites were adjudicated by one rater;
729 the verdicts, evidence and file hashes are published so that the adjudication
730 can be repeated.

731 10.1. Future work

732 Driving forged tokens at the service, under the stock library and the narrow
733 fix, would turn the reachability analysis into an availability measurement,
734 and a fix that removes the attempt for every input without requiring callers
735 to pre-parse keys remains to be designed. Bisecting the OpenSSL history
736 between 3.0 and 3.2 would identify the changes responsible for the cost. A
737 systematic survey of JWT and JOSE libraries, and of other authentication
738 paths that resolve key material on every call, would estimate how often the
739 design choice is made; cross-file call-site resolution and a second raters would
740 strengthen the prevalence evidence. A dedicated x86_64 host, a Linux-native
741 baseline for containerisation and interleaved A/B windows would each close a
742 limitation stated above.

743 11. Conclusion

744 The per-request cost of token validation in `jsonwebtoken` is real and is not
745 the signature: the primitive is 2.208 μ s of a 29.92 μ s call, and key conversion
746 costs 0.583 μ s. The dominant component is a parse attempted in the wrong
747 order, whose failure is constructed and discarded on every request at 18.04 μ s
748 (60% of the call). Its price is set causally by the OpenSSL release a base image
749 happens to pin: the same calls from C cost 394.4 μ s on OpenSSL 3.0.16 and
750 8.447 μ s on 3.5.8. It is a design choice, not a property of JWT: of six other
751 libraries in four languages none attempts the parse. It cannot be removed
752 naively, because it is load-bearing for the library’s key-confusion defence; a
753 narrow fix preserves that defence and makes validation 4.15 times faster, and
754 pre-parsing the key once is both the fastest configuration and the only one
755 no input can push onto the slow path. Yet no call site in our sample does so.

756 The broader lesson for software engineering is about attribution. The
757 explanation that presents itself before measurement was wrong twice, and
758 the component that sets the cost is one most measurement protocols record
759 once per run, if at all. Performance studies should record the identity of the
760 components whose behaviour they measure, including transitive dependencies
761 such as the bundled cryptographic library, on every measurement, at the
762 granularity at which they can change.

763 CRediT authorship contribution statement

764 **Jannatul Ferdous Katha:** Conceptualization, Methodology, Software,
765 Investigation, Data curation, Formal analysis, Visualization, Writing – original

766 draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md**
767 **Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:**
768 Supervision, Project administration, Writing – review & editing. **Dip Nandi:**
769 Supervision, Writing – review & editing.

770 Declaration of competing interest

771 The authors declare that they have no known competing financial interests
772 or personal relationships that could have appeared to influence the work
773 reported in this paper.

774 Funding

775 This research did not receive any specific grant from funding agencies in
776 the public, commercial, or not-for-profit sectors.

777 Data availability

778 The data supporting the results of this study are openly available in the
779 replication package (Katha et al., 2026), archived at Zenodo under <https://doi.org/10.5281/zenodo.23011153> and maintained in a public GitHub
780 repository.³ It contains all measurements, the instrumentation, the analysis
781 code, the cross-library harnesses and the scripts that generate every numeric
782 value and figure in this paper and its supplementary material; none is entered
783 by hand. The survey corpus is third-party source and is not redistributed;
784 a manifest pins every file examined by content hash (502 files across 457
785 repositories).

787 Ethics and responsible disclosure

788 This study measures software behaviour and publicly available source
789 code; it involves no human participants and no personal data. No attack
790 was run against any system the authors do not control. The key-confusion
791 behaviour discussed in Section 8 is an existing, publicly documented attack
792 class tested only through the library’s own test suite; the upstream report

³<https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->

describes the constraint at the level of mechanism and does not include an exploitation recipe.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this manuscript, generative AI tools were used to improve language and readability. The authors reviewed all content following such use and take full responsibility for the contents of the manuscript.

Acknowledgements

The authors acknowledge ELITE Research Lab for providing the computational resources and support necessary for this research.

Appendix A. Supplementary data

Supplementary material related to this article (extended related work, measurement-protocol details, additional analyses for RQ1 to RQ3, the full threats to validity and a design note on relocation) is provided as a separate file, referred to in the text as Supplementary Sections S1 to S7.

References

- Acar, Y., Backes, M., Fahl, S., Garfinkel, S., Kim, D., Mazurek, M.L., Stransky, C., 2017. Comparing the usability of cryptographic APIs, in: 2017 IEEE Symposium on Security and Privacy (SP). doi:10.1109/SP.2017.52.
- Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update for jsonwebtoken. Auth0 security bulletin, 21 December 2022, <https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken>. Accessed 2026-09-28.
- Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages* 1, 52:1–52:27. doi:10.1145/3133876.
- Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and .NET, in: *ECOOP 2007 – Object-Oriented Programming*, Springer. pp. 151–175. doi:10.1007/978-3-540-73589-2_8.
- Chen, J., Shang, W., 2017. An exploratory study of performance regression introducing code changes, in: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). doi:10.1109/ICSME.2017.13.

825 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017.
826 An empirical analysis of the Docker container ecosystem on GitHub, in: 2017
827 IEEE/ACM 14th International Conference on Mining Software Repositories
828 (MSR). doi:10.1109/MSR.2017.67.

829 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web
830 servers. *ACM Transactions on Computer Systems* 24. doi:10.1145/1124153.11
831 24155.

832 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
833 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
834 Proceedings of the 27th USENIX Security Symposium, pp. 343–359.

835 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017. On
836 the (in-)security of JavaScript object signing and encryption, in: Proceedings
837 of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS '17).
838 doi:10.1145/3150376.3150379.

839 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical study
840 of cryptographic misuse in Android applications, in: Proceedings of the 2013
841 ACM SIGSAC Conference on Computer and Communications Security (CCS
842 '13). doi:10.1145/2508859.2516693.

843 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography.
844 pyca/cryptography project statement, <https://cryptography.io/en/latest/statements/state-of-openssl/>. Maintainer statement, not peer reviewed;
845 accessed 2026-09-24.

847 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java perfor-
848 mance evaluation, in: Proceedings of the 22nd Annual ACM SIGPLAN Confer-
849 ence on Object-Oriented Programming Systems, Languages and Applications
850 (OOPSLA '07). doi:10.1145/1297027.1297033.

851 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-2015-
852 9235, GHSA-c7hr-j4mj-j2w6). <https://github.com/advisories/GHSA-c7hr-j4mj-j2w6>. Accessed 2026-09-28.

854 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and detecting
855 real-world performance bugs, in: Proceedings of the 33rd ACM SIGPLAN
856 Conference on Programming Language Design and Implementation (PLDI '12),
857 pp. 77–88. doi:10.1145/2254064.2254075.

858 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT). Request for
859 Comments RFC 7519. Internet Engineering Task Force. doi:10.17487/RFC7519.

860 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
861 Proceedings of the 2013 International Symposium on Memory Management
862 (ISMM '13). doi:10.1145/2464157.2464160.

863 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.,

2014. The promises and perils of mining GitHub, in: Proceedings of the 11th Working Conference on Mining Software Repositories (MSR 2014). doi:10.1145/2597073.2597074.

Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023. Performance evolution of configurable software systems: An empirical study. Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replication package: a pre-authentication key parse dominates JWT validation in jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: <https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->.

Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer practices for locally handling exceptions, in: Proceedings of the 13th International Conference on Mining Software Repositories (MSR '16), pp. 484–487. doi:10.1145/2901739.2903497.

Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An extensible approach to validating the correct usage of cryptographic APIs, in: 32nd European Conference on Object-Oriented Programming (ECOOP 2018), pp. 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

Laaber, C., Leitner, P., 2018. An evaluation of open-source software microbenchmark suites for continuous performance assessment, in: Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18). doi:10.1145/3196398.3196407.

Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the cloud. how bad is it really? Empirical Software Engineering 24. doi:10.1007/s10664-019-09681-1.

Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic software fail? A case study and open problems, in: Proceedings of the 5th Asia-Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.2637237.

Leitner, P., Cito, J., 2016. Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. ACM Transactions on Internet Technology 16. doi:10.1145/2885497.

Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee reproducibility. arXiv preprint arXiv:2601.12811.

Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09). doi:10.1145/1508244.1508275.

Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:

903 Proceedings of the 38th International Conference on Software Engineering (ICSE
904 '16). doi:10.1145/2884781.2884790.

905 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang, Z., Yuan,
906 X., 2020. Performance evaluation and modeling of cryptographic libraries for
907 MPI communications. arXiv preprint arXiv:2010.06139.

908 Nistor, A., Jiang, T., Tan, L., 2013a. Discovering, reporting, and fixing performance
909 bugs, in: Proceedings of the 10th Working Conference on Mining Software
910 Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.

911 Nistor, A., Song, L., Marinov, D., Lu, S., 2013b. Toddler: Detecting perfor-
912 mance problems via similar memory-access patterns, in: Proceedings of the
913 35th International Conference on Software Engineering (ICSE '13), pp. 562–571.
914 doi:10.1109/ICSE.2013.6606602.

915 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
916 optimization in Java. ACM Transactions on Programming Languages and
917 Systems 28. doi:10.1145/1111596.1111598.

918 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantarcioglu,
919 M., Yao, D.D., 2019. CryptoGuard: High precision detection of cryptographic
920 vulnerabilities in massive-sized Java projects, in: Proceedings of the 2019 ACM
921 SIGSAC Conference on Computer and Communications Security (CCS '19), pp.
922 2455–2472. doi:10.1145/3319535.3345659.

923 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance profiler
924 for optimizing the order of evaluations, in: Proceedings of the 26th ACM
925 SIGSOFT International Symposium on Software Testing and Analysis (ISSTA
926 '17). doi:10.1145/3092703.3092716.

927 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in JavaScript:
928 An empirical study, in: Proceedings of the 38th International Conference on
929 Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/2884781.2884829.

930 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current Practices.
931 Request for Comments RFC 8725. Internet Engineering Task Force. doi:10.174
932 87/RFC8725.

933 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead of
934 post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th
935 International Conference on Emerging Networking EXperiments and Technologies
936 (CoNEXT '20). doi:10.1145/3386367.3431305.

937 Song, L., Lu, S., 2014. Statistical debugging for real-world performance problems,
938 in: Proceedings of the 2014 ACM International Conference on Object Oriented
939 Programming Systems Languages & Applications (OOPSLA '14), pp. 561–578.
940 doi:10.1145/2660193.2660234.

941 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vulnerabilities

942 in JavaScript-based web servers, in: Proceedings of the 27th USENIX Security
943 Symposium, pp. 361–376.

944 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L., 2023.
945 JWTKey: Automatic cryptographic vulnerability detection in JWT applications,
946 in: Computer Security – ESORICS 2023, Springer. doi:10.1007/978-3-031-5
947 1479-1_14.

948 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang,
949 B., 2026. Token time bomb: Evaluating JWT implementations for vulnerability
950 discovery, in: Proceedings of the Network and Distributed System Security
951 Symposium (NDSS '26).

952 Zaman, S., Adams, B., Hassan, A.E., 2012. A qualitative study on performance
953 bugs, in: Proceedings of the 9th IEEE Working Conference on Mining Software
954 Repositories (MSR '12), pp. 199–208. doi:10.1109/MSR.2012.6224281.

955 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On the
956 relation between outdated Docker containers, severity vulnerabilities, and bugs,
957 in: Proceedings of the 26th IEEE International Conference on Software Analysis,
958 Evolution and Reengineering (SANER '19), pp. 491–501. doi:10.1109/SANER.
959 2019.8668013.

960 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P.,
961 Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting overheads
962 of service mesh sidecars, in: Proceedings of the 2023 ACM Symposium on Cloud
963 Computing (SoCC '23). doi:10.1145/3620678.3624652.